Alexandria University — Faculty of Engineering
Department of Computer Engineering

Using GPU in Matrix Operations

CPU vs **GPU**

Students: Samaa Ibrahim El-Hakmy 22010820
Nour-Eldin Akram Elsayed 22011309

Alexandria University, Faculty of Engineering — Spring 2026

1. Introduction

Modern computing workloads—especially in machine learning, scientific simulation, and computer vision—are dominated by large-scale linear algebra. Matrix multiplication in particular is an $\mathcal{O}(n^3)$ operation whose cost grows rapidly with size. Graphics Processing Units (GPUs) offer massively parallel execution and high memory bandwidth that can dramatically reduce this cost compared to a traditional Central Processing Unit (CPU).

This lab explores that gap concretely. We implement the computation

$$D = A \times B \times C + A$$

where $A, B, C \in R^{n \times n}$, benchmark it with **NumPy** (CPU) and **CuPy** (GPU/CUDA) across nine matrix sizes from 128×128 to $10{,}000 \times 10{,}000$, and quantify the resulting speedup.

2. Methodology

2.1. Libraries

- **NumPy 1.x** — the standard Python array library, executing on a single CPU core using BLAS routines.
- **CuPy** — a NumPy-compatible library that dispatches operations to an NVIDIA GPU via CUDA. CuPy's `@` operator maps directly to `cuBLAS SGEMM`.

2.2. Benchmarking Protocol

For each matrix size $n \in \{128, 256, 512, 1024, 1536, 2048, 4000, 8000, 10000\}$:

1. Allocate fresh random matrices A, B, C of shape $n \times n$.
2. Execute $D = A \times B \times C + A$ and record wall-clock time.
3. Repeat 10 times; report the **mean** elapsed time.
4. GPU timing uses `cp.cuda.Event` for device-accurate measurement.

3. Implementation

3.1. Full Source Code

Listing 1: Complete benchmarking script

```python
import numpy as np
import cupy as cp
import time
import matplotlib.pyplot as plt

SIZES   = [128, 256, 512, 1024, 1536, 2048, 4000, 8000, 10000]
REPEATS = 10

def benchmark_cpu(n):
    times = []
    for _ in range(REPEATS):
        A = np.random.rand(n, n)
        B = np.random.rand(n, n)
        C = np.random.rand(n, n)
        start = time.perf_counter()
        D = A @ B @ C + A
        end   = time.perf_counter()
        times.append(end - start)
    return np.mean(times)

def benchmark_gpu(n):
    times = []
    for _ in range(REPEATS):
        A = cp.random.rand(n, n, dtype=cp.float32)
        B = cp.random.rand(n, n, dtype=cp.float32)
        C = cp.random.rand(n, n, dtype=cp.float32)

        start = cp.cuda.Event()
        stop  = cp.cuda.Event()
        start.record()
        D = A @ B @ C + A
        stop.record()
        stop.synchronize()

        gpu_time_ms = cp.cuda.get_elapsed_time(start, stop)
        times.append(gpu_time_ms / 1000.0)

        del A, B, C, D
        cp.get_default_memory_pool().free_all_blocks()
    return np.mean(times)
```

3.2. Key Design Decisions

- **CUDA Events for GPU timing.** `time.perf_counter()` measures host-side wall time which may not capture asynchronous GPU execution. CUDA events are placed directly on the GPU stream, giving accurate device-side elapsed time.
- **Memory pool reset.** CuPy uses a memory pool by default. Calling `free_all_blocks()` after each trial prevents cached blocks from flattering later allocations and ensures fair per-trial measurement.
- **Averaging over 10 runs.** A single run is noisy due to OS scheduling, cache warm-up, and CUDA JIT compilation on the first call. The mean of 10 trials yields a stable estimate.

4. Results

4.1. Raw Benchmark Data

Table 1: Benchmark results for all tested matrix sizes

Matrix Size	CPU Time (s)	GPU Time (s)	Speedup
128×128	0.000486	0.005921	0.08×
256×256	0.003224	0.000113	28.48×
512×512	0.004777	0.000319	14.99×
1024×1024	0.022501	0.001095	20.55×
1536×1536	0.067980	0.003041	22.35×
2048×2048	0.150264	0.007616	19.73×
4000×4000	1.073150	0.063962	16.78×
8000×8000	7.704448	0.577311	13.35×
10000×10000	15.117782	0.721720	20.95×

5. Analysis and Discussion

5.1. Small Matrices: GPU Overhead Dominates ($n = 128$)

At $n = 128$ the GPU is **slower** than the CPU (speedup < 1). Launching a CUDA kernel, transferring control to the GPU, and synchronizing the device all incur fixed overhead in the range of milliseconds. The actual computation ($128^3 \approx 2 \times 10^6$ FLOPs) is trivially small compared to this overhead, so the CPU finishes first.

5.2. Rapid Rise in Speedup ($n = 256$–1536)

Once the problem is large enough to saturate the GPU's streaming multiprocessors, speedup jumps sharply—peaking near **28**× at $n = 256$ and stabilising around 20× through $n = 1536$. The GPU executes thousands of independent multiply-accumulate operations in parallel, while the CPU is constrained to its core count and BLAS threading model.

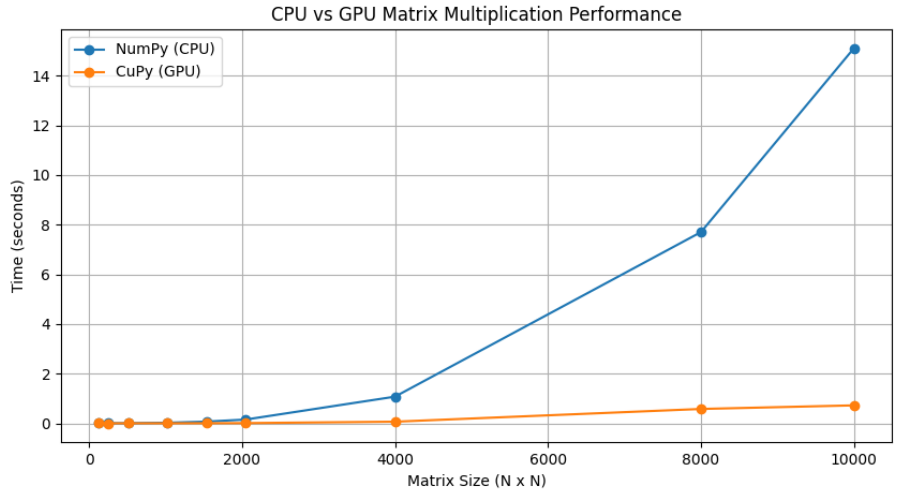

Figure 1: Absolute execution time for NumPy (CPU) vs. CuPy (GPU) as a function of matrix size. The CPU curve grows steeply beyond $n = 4000$, while the GPU curve remains nearly flat on this scale.

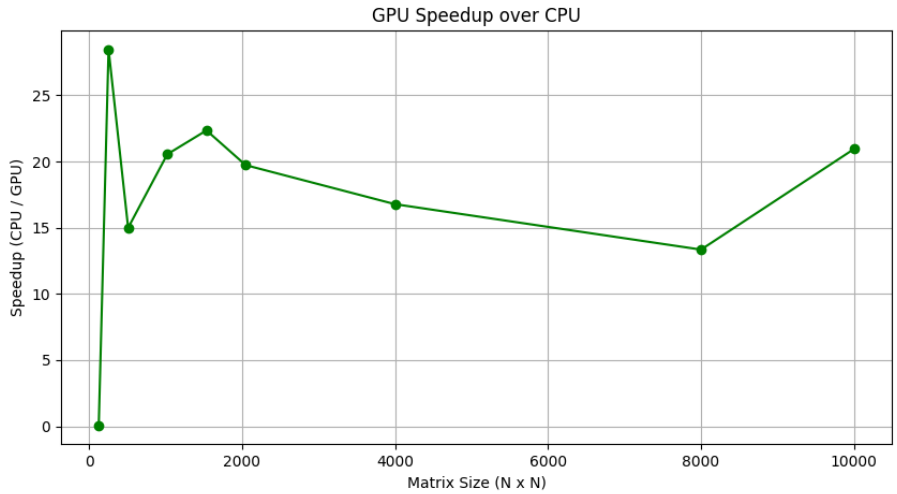

Figure 2: GPU speedup ratio (CPU time / GPU time) across matrix sizes. The curve shows a sharp rise at small sizes (GPU parallelism kicks in), a plateau in the mid-range, a temporary dip at $n = 8000$ (memory bandwidth pressure), and a recovery at $n = 10000$.

5.3. Mid-Range Plateau ($n = 2048$–4000)

Speedup gradually decreases from $\approx 22\times$ to $\approx 17\times$ as the matrices grow past what fits comfortably in L2/L3 GPU cache. Memory traffic begins to limit throughput on both platforms, though the GPU's high-bandwidth memory (HBM) maintains a clear advantage.

5.4. Large Matrix Dip and Recovery ($n = 8000$–10000)

At $n = 8000$ speedup drops to $\approx 13\times$. A single 8000×8000 `float32` matrix consumes ≈ 244 MB; holding three such matrices simultaneously pushes the GPU VRAM close to its limit, increasing eviction and re-transfer costs.

Interestingly, speedup **recovers** to $\approx 21\times$ at $n = 10000$. A likely explanation is that the CPU suffers even more from its limited last-level cache at this scale, making the CPU-over-GPU penalty worse in absolute terms and thereby lifting the ratio.

6. Conclusion

This experiment demonstrates quantitatively that GPU-accelerated matrix multiplication via CuPy offers substantial speedups over CPU-based NumPy for matrix sizes commonly encountered in engineering and machine learning workloads. The speedup is not monotonic—GPU overhead dominates at tiny sizes, and memory pressure reduces efficiency at extreme sizes—but in the practically important range ($n \approx 1000$–5000), the GPU delivers a consistent $15\times$–$22\times$ speedup.